

带证据的知识图谱校验 Evidence-Carrying Validation for Knowledge Graphs

本篇范围说明：这是一篇 arXiv 论文（Gabe Fierro，科罗拉多矿业学院）。下面「全文翻译」一节覆盖摘要、引言、「缺失的校验接口」那一章、证据语义的核心设计、以及评测与案例研究的主要发现；形状代数的形式化定义、Python 投影接口的逐个方法签名、以及暖通空调那个契约的领域细节不逐段翻译。

核心内容

Q：这篇论文要解决什么问题？

A：程序在消费一份不是自己维护的知识图谱时，需要知道这份图里有没有它这个任务要用的信息。拿一份「形状」（schema）去校验这张图可以回答这个问题，但现有的校验接口通常只返回一个合规位，或者一份面向失败的报告——它既不说清检查为什么通过，也不说清失败背后那些部分匹配是什么样。

Q：现有接口具体差在哪？举个能说明问题的例子。

A：论文举的是 SHACL（W3C 的图形状校验语言）。它能定位违规，但既不枚举通过的焦点节点，也不要求嵌套证据足以论证每一次失败。后果是：

一份 SHACL 报告里没有某个节点的结果，对消费方来说是有歧义的：这个节点可能通过了，可能它的目标选择器漏掉了它，也可能求值器压根没检查它。

这是这篇论文最锋利的一句话。三种完全不同的情况产出同一个观测——什么都没有。

而且即使有一条明确的「通过」，它也不指出那个让检查通过的子图是什么，而应用需要消费的正是那个子图。

Q：作者提的方案是什么？

A：叫「带证据的校验」。校验器不再只返回真假：

结果	返回什么	里面有什么
节点满足形状	满足轨迹（satisfaction trace）	选中的焦点、被求值的约束、决定性的布尔与基数证据、支撑图路径的具体三元组

结果	返回什么	里面有什么
节点不足	失败见证 (failure witness)	同上，加上「可达的候选」与「缺失的义务」

两者遵循同一套形状代数，并且通过「否定」变成互递归——一条轨迹里含着它孩子的见证，一个见证里含着它孩子的轨迹。

为什么否定会导致互递归：如果一条约束是「不满足形状 S」，那么这条约束通过（产出一条轨迹）恰恰是因为它的子检查失败了（产出一个见证）。所以两种对象必须能互相嵌套。

Q：那个叫「覆盖」的东西是什么，为什么它是关键？

A：报告必须枚举出所有被选中的「节点—形状」检查对，并给每一对附上一条明确的通过或失败记录。这个枚举叫报告的「覆盖」。

它解决的正是上面那个歧义：有了覆盖，消费方就能区分「通过了」「目标选择器没选中它」「压根没求值」这三种情况——因为前者会出现在覆盖里并带一条通过记录，后两者不出现在覆盖里。

Q：带证据的代价有多大？

A：这是论文最实用的一部分数字。在两个真实的形状图语料上（Brick 和 ASHRAE 223P 两个楼宇语义模型标准）：

模式	相对「只判合规」的成本
全对物化（把每一对检查的规范证据都算出来）	Brick 2.07×、223P 1.54×（中位数）
只发现失败并按需解释每个失败对	Brick 1.06×、223P 1.10×
无失败的 223P 模型，只做发现	1.01×

为什么按需模式便宜那么多：因为失败很稀少——Brick 的 247,333 对选中检查里只有 3.1% 失败，223P 的 228,760 对里只有 3.9% 失败。所以「只解释失败的那些」把成本从 2 倍压到了 1.06 倍。

而按需的开销随失败比例上升——这一条论文用一张图给了：横轴是失败对的百分比，纵轴是按需模式的开销倍数。

Q：拿到证据之后，程序怎么用它？

A：论文的案例研究是一个暖通空调的应用契约（APAR），它要求混合风、回风、室外风三处温度输入，并且都要声明单位。

消费方做的事分两步，而这两步分别用了两种证据：

用失败见证定位问题：失败的类型化投影指出 `mat1` 是一个可达但被拒的候选，并报告缺 1 个值。

用通过的轨迹选修法：通过的那两条轨迹投影出两个华氏度的值及其支撑路径——于是消费方知道这栋楼的单位约定是华氏度。

最后它提出的修补是 `(mat1, hasUnit, °F)`：用失败定位该改哪里，用通过的语句决定改成什么值。

这个组合是全篇最实用的一点：失败告诉你缺什么，通过告诉你按什么惯例补。

背景补充 / 点评

先解释两个术语，它们是读懂这篇的前提。

SHACL 是 W3C 定的一套「图形状约束语言」。它的作用类似关系数据库的表结构约束，但对象是图：你写一份形状，说「凡是类型为空调机组的节点，必须有三个温度输入，每个输入必须声明单位」，然后拿这份形状去校验一张图，看它合不合规。

焦点节点 (focus node) 指校验时正在被检查的那个节点。一份形状会先用「目标选择器」挑出一批焦点节点，再对每个焦点逐条查约束。「节点—形状对」就是「哪个节点被哪份形状检查过」这件事。

再说这篇论文和我手上项目的关系，因为它对上的东西比我预想的多。

prove 那个项目有一道叫 gates 的机制：回答里出现的每个数字都必须能在证据表里找到对应的行。这篇论文做的是同一件事的校验版本——不是「答案能不能追溯到证据」，而是「校验结论能不能追溯到证据」。

而它指出的那个歧义，我在 8 月 14 日记过同一个形状。当时的表述是：

「拒答是对的」有两种，而它们的后果完全不同：**设计上拒答**（这个问题本来就不该被回答）不该出现在缺口清单里，它是健康行为；**能力缺口**（问题合理、只是我们还做不到）该出现而且要一直显眼。而它们现在产出的信号一模一样。

这篇论文里是三种情况产出同一个观测（通过了、没被选中、没求值），我那次是两种情况产出同一个信号。同一个病：一个字段回答了两个问题，而只有其中一个需要人去做事。

它给的解法比我当时的解法更一般。我当时的做法是加一档（`known_fail`），让那两种拒答落在不同的档上。这篇的做法是要求「覆盖」——把所有被检查过的对都枚举出来，于是「不在覆盖里」和「在覆盖里且失败」天然区分开。加档是针对已知的两种情况打补丁，枚举覆盖是让这类歧义整体消失。

第三处对照，也是我觉得最该抄的一处：那个「 $1.06\times$ 」的数字。

论文的核心权衡是「带证据要多花多少」。而它给出的答案是：**全量物化贵一倍，但只解释失败几乎不要钱（1.06 倍）——因为失败本来就稀少（3~4%）。**

这个结构在 **prove** 上完全一样：十问验收、评测集、gates 对账，绝大多数情况是通过的。所以「给每次通过都留完整证据」这件事的性价比很低，而「给每次失败都留完整证据」几乎免费。

而我现在的做法偏向前者——gates 每次都把证据表带上。按这篇的思路，通过时只留一个指针（哪条指标、哪些参数），失败时才物化完整证据，成本会明显下降。这条我要记下来，但不是今天动手的事。

这篇论文的留白与该保持怀疑的地方。

第一，那两个语料都是楼宇领域的。 Brick 和 ASHRAE 223P 都是楼宇控制的语义模型标准，而论文自己也说这个方法的动机来自楼宇控制和水处理系统。「失败只占 3~4%」这个前提在别的领域不一定成立——一个正在被大模型批量写入的图，失败比例可能高得多，那按需模式的优势就没了（论文那张右图也说明开销随失败比例上升）。

第二，它没有报证据对象本身的体积。 满足轨迹要保留「支撑图路径的具体三元组」，那么一次全量校验产出的证据总量是多少？论文报了时间开销（1.54~2.07 倍），没报空间开销。对要把证据落盘或者塞进提示词的场景，这个数字是必须知道的。

第三，「未来工作」那一节提到的那件事其实是这篇论文最有价值的应用方向，但它只是提了一下。 原文说：他们此前对大模型修补图的评估发现，**修补质量取决于提示词怎么构造——最有效的提示词把「被违反的约束」和「图里相关的上下文片段」组合在一起。**而组装那个片段目前是一个临时的检索过程。

带证据的校验正好把这件事变成机械的——失败见证里已经含着约束和支撑三元组，直接就是那个提示词该有的内容。但论文没有做这个实验，只是说了方向。

对我自己的启示。除了上面那三处对照，有一条是关于用户画像那个项目的。

画像的校验也会遇到同一个歧义。假设我们定了「一条画像陈述必须有 method 和 evidence 两个字段之一」这条约束，然后拿它去校验存量画像。**如果校验器只报违规，那「某条画像不在报告里」就有三种可能：**它合规；它的类型没被选择器选中（比如那条画像的维度是新加的、形状还没覆盖）；或者校验器因为某个原因跳过了它。

而第二种情况最危险——它看起来像「合规」，实际是「没检查」。这正是 8 月 20 日调研时那个「离线刷回来的数据 trace 很难标」问题的镜像：那次担心的是数据侧缺字段，这次是校验侧漏检查。

所以画像的校验一开始就该要求「覆盖」：报告要枚举「哪条画像被哪份形状检查过」，而不只是列出违规。这是一条现在就能定下来的接口约束，代价很小，事后补很贵。

文中金句

"An absent SHACL result is consequently ambiguous to a consumer: the node may have passed, its target selector may have missed it, or the evaluator may never have checked it." 因此，一份 SHACL 报告里的「结果缺席」对消费方来说是有歧义的：这个节点可能通过了，可能它的目标选择器漏掉了它，也可能求值器压根没检查它。

"Most graph validation interfaces discard the information used to compute conformance."

大多数图校验接口把用来算出「是否合规」的那些信息丢掉了。

"Instead, validation should preserve what the validator learned, and present that information in a form another program can use." 校验应当保留校验器学到的东西，并以另一个程序能用的形式呈现它。

"The consumer proposes (mat1, hasUnit, ° F), using the failure to locate the edit and the passing statements to choose its value." 消费方提出把 (mat1, hasUnit, °F) 补进去——用失败来定位该改哪里，用通过的语句来决定改成什么值。

全文翻译

摘要

Programs that consume a knowledge graph they do not maintain, such as applications, authoring platforms, and LLM agents, need to know whether the graph contains the information their task requires.

那些消费一份自己并不维护的知识图谱的程序——应用、创作平台、大模型智能体——需要知道这份图里是否含有它们这个任务所要求的信息。

Validating the graph against a schema can answer this question, but existing validation interfaces usually return a conformance bit or failure-oriented report without identifying why checks pass or the partial matches behind failures.

拿一份 schema 去校验这张图可以回答这个问题，**但现有的校验接口通常只返回一个合规位、或者一份面向失败的报告，既不指出检查为什么通过，也不指出失败背后的那些部分匹配。**

We present an evidence-carrying validation interface: every selected node - shape check returns either a satisfaction trace or failure witness. These are mutually recursive objects that retain constraints, cardinality decisions, paths, and supporting triples.

我们提出一个**带证据的校验接口**：每一个被选中的「节点—形状」检查，要么返回一条**满足轨迹**，要么返回一个**失败见证**。这两者是**互递归**的对象，它们保留约束、基数判定、路径、以及支撑用的三元组。

We implement this interface in Shifty, an experimental SHACL validator. Against two real-world shape graph corpora, materializing all-pair evidence costs a median 1.54 - 2.07 × conformance-only validation. A case study then shows how programs combine passing and failing evidence to diagnose missing information and guide repair.

我们在 Shifty（一个实验性 SHACL 校验器）里实现了这个接口。在两个真实的形状图语料上，**把所有检查过的证据全部物化，中位成本是「只判合规」的 1.54 到 2.07 倍**。随后一个案例研究展示了程序如何**把通过的证据与失败的证据组合起来**，诊断缺失信息并指导修补。

一、引言：约束和图一起在演化

Knowledge graphs increasingly serve programs that do not maintain them: applications configure themselves from graph contents, platforms author and extend graphs, and LLM agents query and repair them.

知识图谱越来越多地服务于那些并不维护它们的程序：应用从图的内容里自我配置，平台创作和扩展图，大模型智能体查询和修补图。

The constraints on a graph evolve alongside it: maintainers add and revise requirements in the graph's ontology, and new use cases demand structure and metadata the graph does not yet carry.

图上的约束和图本身一起演化：维护者在图的本体里增加和修订要求，而新的使用场景会要求图还没有携带的结构与元数据。

Shapes make these expectations testable: validation of a graph against a shape determines semantic sufficiency—whether the current graph contains enough structure and metadata to configure and execute an application.

形状让这些期望变得可测：拿形状校验一张图，决定的是**语义充分性**——当前这张图是否含有足够的结构与元数据来配置并执行一个应用。

Acting on a validation result requires specifics. On success, the application needs the concrete nodes and graph paths that satisfy the contract. On failure, an operator or repair program needs the reachable candidates and missing obligations that explain what the graph lacks.

要对一个校验结果采取行动，需要具体信息。成功时，应用需要那些满足契约的具体节点与图路径。失败时，操作员或修补程序需要那些**可达的候选与缺失的义务**，来解释这张图缺了什么。

Existing validation interfaces provide neither view completely. SHACL localizes violations but does not enumerate passing focus nodes or require nested evidence sufficient to justify each failure. An absent SHACL result is consequently ambiguous to a consumer: the node may have passed, its target selector may have missed it, or the evaluator may never have checked it. Even an explicit pass does not identify the validating subgraph that an application should consume.

现有的校验接口这两个视图都没给全。SHACL 能定位违规，但**不枚举通过的焦点节点，也不要求足以论证每一次失败的嵌套证据**。因此，一份 SHACL 报告里的结果缺席对消费方来说是有歧义的：这个节点可能通过了，可能它的目标选择器漏掉了它，也可能求值器压根没检查它。而且即使有一条明确的通过，它也不指出应用该去消费的那个「使校验通过的子图」是什么。

论文的三项贡献：

- a validation result interface that exposes target coverage, both result polarities, result-relative complete justification, and concrete graph support; 一个校验结果接口，它暴露**目标覆盖、两种结果极性、相对于结果完整的论证、以及具体的图支撑**；
- a polarity-aware evidence semantics with mutually recursive failure witnesses and satisfaction traces; 一套**极性感知的证据语义**，其中失败见证与满足轨迹互递归；
- an implementation that exposes typed Python result objects, preserves authored identities through normalization, and materializes evidence completely or on demand. 一个实现，它暴露带类型的 Python 结果对象、在规范化过程中保留作者写下的身份、并且可以完整地或按需地物化证据。

二、缺失的那个校验接口

Most graph validation interfaces discard the information used to compute conformance. A validation report may identify a violation while omitting which other nodes were selected, what made their checks pass, or how closely a candidate matched before it failed.

大多数图校验接口把用来算出「是否合规」的信息丢掉了。一份校验报告可能指出了一处违规，却漏掉了这些：还有哪些节点被选中了、是什么让它们的检查通过了、以及一个候选在失败之前匹配到了什么程度。

Recovering these facts can be expensive, requiring rerunning or even reverse-engineering the validator. Instead, validation should preserve what the validator learned, and present that information in a form another program can use.

把这些事实找回来是昂贵的，需要重新运行、甚至逆向工程那个校验器。**校验应当保留校验器学到的东西，并以另一个程序能用的形式呈现它。**

A validation result object forms the boundary between the validator and its consumers. Its contents determine whether a consumer can explain a result, extract conforming data, diagnose target selection, or propose a repair without invoking the validator again.

一个校验结果对象构成校验器与它的消费方之间的边界。它的内容决定了消费方能不能在**不再次调用校验器**的前提下：解释一个结果、抽取合规的数据、诊断目标选择、或者提出一个修补。

When a report contains no failure for a node, a consumer must be able to distinguish a passing check from a missed target or an unevaluated check. The report therefore needs to enumerate the selected node - shape checks and attach an explicit passing or failing record to each one. We call this enumeration the report's coverage.

当一份报告里某个节点没有失败记录时，**消费方必须能区分「检查通过了」、「目标被漏掉了」、和「检查没被求值」**。因此报告需要**枚举出被选中的那些「节点—形状」检查**，并给每一个附上一**条明确的通过或失败记录**。我们把这个枚举叫做报告的「覆盖」。

The result record should also preserve the constraint evaluations that establish the pass or failure and identify the input triples.

结果记录还应当保留那些[确立了通过或失败的约束求值过程](#)，并指明输入的三元组。

三、证据语义：为什么两种对象必须互递归

论文把满足轨迹与失败见证建立在同一套[形状代数](#)上，两者的字段结构对称：都保留选中的焦点、被求值的约束、决定性的布尔与基数证据、以及支撑图路径的具体三元组。

Both objects follow a common shape algebra and become mutually recursive through negation: a trace contains its child's witness or a witness contains its child's trace.

两种对象遵循同一套形状代数，并且[通过否定变成互递归](#)：一条轨迹里含着它孩子的见证，或者一个见证里含着它孩子的轨迹。

Each object retains the selected focus, evaluated constraint, decisive Boolean and cardinality evidence, and the concrete triples supporting graph paths. The result is a pruned, machine-interpretable justification.

每个对象保留：选中的焦点、被求值的约束、决定性的布尔与基数证据、以及支撑图路径的具体三元组。[结果是一份剪枝过的、机器可解释的论证。](#)

四、评测：证据物化的成本

We ran each mode ten times per model on one core of an Apple M2 MacBook Air with 24 GB of RAM. Parsing, SHACL-AF inference, normalization, and indexing run once per model, untimed, so the ratios isolate evidence construction.

我们在一台 24 GB 内存的 Apple M2 MacBook Air 的单核上，对每个模型的每种模式各跑十次。解析、SHACL-AF 推理、规范化和建索引每个模型只做一次且不计，[所以这些比值隔离出的正是「构造证据」这一部分的开销。](#)

The harness rotates three modes—conformance-only validation, eager all-pair canonical evidence, and failure discovery with on-demand explanation of every failing pair—and checks that each on-demand explanation exactly matches its all-pair counterpart.

测试装置轮换三种模式——只判合规、急切地算出所有检查对的规范证据、以及「发现失败并按需解释每一个失败对」——[并且检查每一条按需解释与它在全对模式下的对应结果完全一致。](#)

Each model contributes the median of its paired per-round ratios; across models, all-pair materialization costs a median $2.07\times$ conformance-only execution on Brick and $1.54\times$ on 223P. Only 3.1% and 3.9% of the 247,333 Brick and 228,760 223P selected node - shape checks fail, so discovering and explaining only failures reduces the ratios to $1.06\times$ and $1.10\times$.

每个模型贡献它逐轮成对比值的中位数；跨模型看，**全对物化在 Brick 上的中位成本是「只判合规」的 2.07 倍，在 223P 上是 1.54 倍**。而 Brick 的 247,333 对与 223P 的 228,760 对被选中的「节点—形状」检查中，只有 3.1% 和 3.9% 失败，所以只发现并解释失败，把比值降到了 1.06 倍和 1.10 倍。

For the 223P model with no failures, discovery costs $1.01 \times$ conformance, while overhead rises with the failed-pair fraction.

对于那个没有失败的 223P 模型，**只做发现的成本是合规校验的 1.01 倍**；而开销随失败对的比例上升。

五、案例研究：把通过与失败组合起来选修法

The APAR contract requires mixed-, return-, and outside-air temperature inputs with declared units.

APAR 这个契约要求混合风、回风、室外风三处温度输入，**并且都要声明单位**。

The consumer locates the incomplete point and cardinality deficit from the failure, then obtains the building's unit convention from the passing statements. The consumer obtains one failure and two satisfactions for `ahu1` without walking internal evaluator state.

消费方**从失败里定位到那个不完整的测点与基数缺口**，然后从通过的语句里得到这栋楼的单位约定。它对 `ahu1` 拿到一个失败与两个满足，**全程不需要去遍历校验器的内部状态**。

The failure's typed projections identify `mat1` as a reachable rejected candidate and report one missing value; the satisfactions project the two Fahrenheit values and their supporting paths. The consumer proposes `(mat1, hasUnit, ° F)`, using the failure to locate the edit and the passing statements to choose its value.

失败的类型化投影指出 `mat1` 是一个**可达但被拒的候选**，并报告缺 1 个值；那些满足则投影出两个华氏度的值及其支撑路径。消费方于是提出把 `(mat1, hasUnit, °F)` 补进去——用失败来定位该改哪里，用通过的语句来决定改成什么值。

六、未来工作

Evidence-carrying validation makes result lineage part of the validator's contract: execution may reorder and share work, but it must retain enough information to construct the specified evidence.

带证据的校验把「结果的来源脉络」变成了校验器契约的一部分：执行过程可以重排、可以共享计算，但它必须保留足够的信息来构造出规定的那份证据。

Platforms like BuildingMOTIF already run validation in a loop during model authoring: shapes describe application requirements, and each failure indicates what to add next. The platform

currently reconstructs that guidance from a failure-oriented report, recomputing facts the validator discarded.

像 BuildingMOTIF 这样的平台在模型创作过程中已经在循环里跑校验了：形状描述应用需求，每一次失败指出下一步该加什么。[但这个平台目前是从一份面向失败的报告里重建那份指引的，重新计算校验器已经丢弃的事实。](#)

Our prior evaluation of LLM-based graph repair found that repair quality depends on prompt construction: the most effective prompts combine the violated constraint with the relevant contextual fragment of the graph. Assembling that fragment is currently an ad hoc retrieval process.

我们此前对基于大模型的图修补做的评估发现：[修补质量取决于提示词怎么构造——最有效的提示词把「被违反的那条约束」与「图里相关的上下文片段」组合在一起。](#)而组装那个片段目前是一个临时的检索过程。

参考链接

- [SHACL \(W3C 形状约束语言\)](#) — 本文的对照基线，也是它指出歧义的那个规范
- [Brick Schema](#) — 评测用的两个真实语料之一，楼宇设备的语义模型标准
- [ASHRAE 223P](#) — 评测用的另一个语料，暖通空调领域的语义标准
- [BuildingMOTIF](#) — 论文提到的那个「在创作循环里跑校验」的平台，是这套接口最直接的消费方